



Lecture 14 (Data Structures 1)

Disjoint Sets

CS61B, Spring 2026 @ UC Berkeley

Josh Hug and Kay Ousterhout

Big O and Big Theta Review

Lecture 14, CS61B, Spring 2026

Big O and Big Theta Review

Introduction to Disjoint Sets

- Disjoint Sets API
- Tracking Connected Components

Implementations

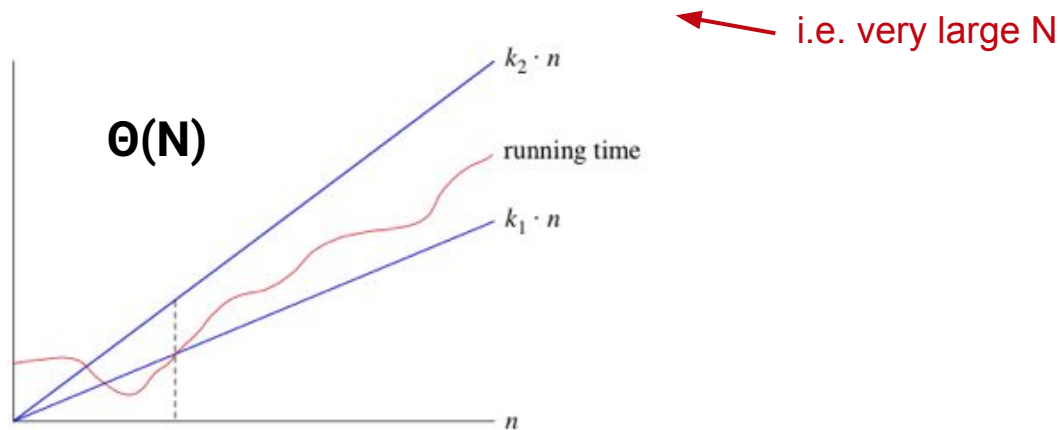
- List of Sets
- Quick Find
- Quick Union
- Weighted Quick Union
- WQU with Path Compression

$$R(N) \in \Theta(f(N))$$

means there exist positive constants k_1 and k_2 such that:

$$k_1 \cdot f(N) \leq R(N) \leq k_2 \cdot f(N)$$

for all values of N greater than some N_0 .



$$R(N) \in \Theta(f(N))$$

means there exist positive constants k_1 and k_2 such that:

$$k_1 \cdot f(N) \leq R(N) \leq k_2 \cdot f(N)$$

for all values of N greater than some N_0 .  i.e. very large N

$$R(N) \in O(f(N))$$

means there exist positive constants k_1 and k_2 such that:

$$R(N) \leq k_2 \cdot f(N)$$

for all values of N greater than some N_0 .  i.e. very large N

Big O is just an upper bound!

Big Theta vs. Big O

	Informal meaning:	Family	Family Members
Big Theta $\Theta(f(N))$	Order of growth is $f(N)$.	$\Theta(N^2)$	$N^2/2$ $2N^2$ $N^2 + 38N + N$
Big O $O(f(N))$	Order of growth is less than or equal to $f(N)$.	$O(N^2)$	$N^2/2$ $2N^2$ $\lg(N)$

Big O is useful when the runtime varies depending on the input
(We'll see this today!)

Disjoint Sets API

Lecture 14, CS61B, Spring 2026

Big O and Big Theta Review

Introduction to Disjoint Sets

- Disjoint Sets API
- Tracking Connected Components

Implementations

- List of Sets
- Quick Find
- Quick Union
- Weighted Quick Union
- WQU with Path Compression

Meta-goals of the Coming Lectures: Data Structure Refinement

Next couple of weeks: Deriving classic solutions to interesting problems, with an emphasis on how sets, maps, and priority queues are implemented.

Today: Deriving the “Disjoint Sets” data structure for solving the “Dynamic Connectivity” problem. We will see:

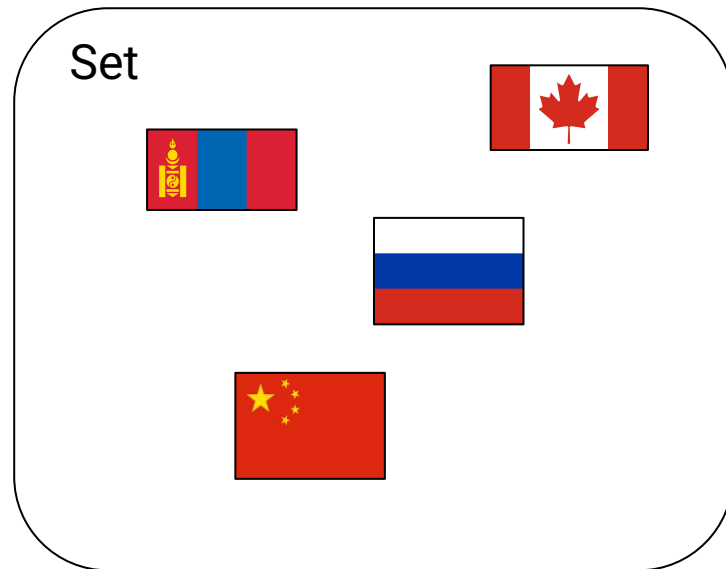
- How a data structure design can evolve from basic to sophisticated.
- How our choice of underlying abstraction can affect asymptotic runtime (using our formal Big-O and Big-Theta notation) and code complexity.



A Set is a collection of elements with no duplicates

Example:

- `add(Russia)`
- `add(Mongolia)`
- `add(China)`
- `add(Mongolia)` ← *Already in the set, so nothing happens*
- `add(Canada)`
- `contains(Mongolia)?` **true**



Set operations are $O(1)$

(We'll learn why later in the course)

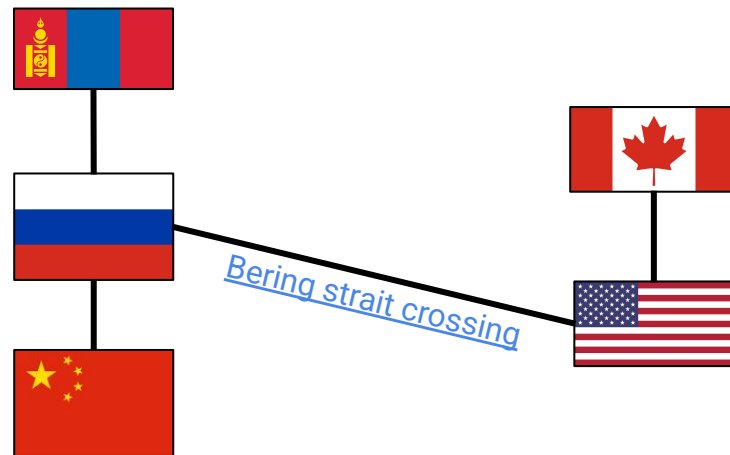
The Disjoint Sets Data Structure

The Disjoint Sets data structure has two operations:

- `connect(x, y)`: Connects x and y.
- `isConnected(x, y)`: Returns true if x and y are connected. Connections can be transitive, i.e. they don't need to be direct.

Example:

- `connect(Russia, China)`
- `connect(Russia, Mongolia)`
- `isConnected(China, Mongolia)?` **true**
- `connect(USA, Canada)`
- `isConnected(USA, Mongolia)?` **false**
- `connect(Russia, USA)`
- `isConnected(USA, Mongolia)?` **true**



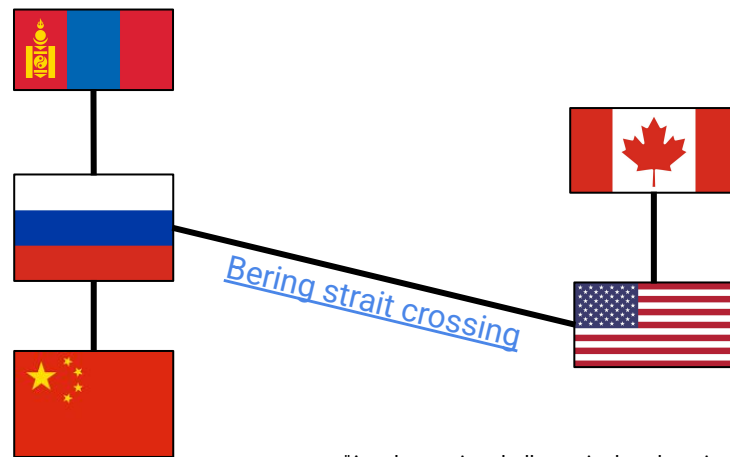
The Disjoint Sets Data Structure

The Disjoint Sets data structure has two operations:

- `connect(x, y)`: Connects x and y .
- `isConnected(x, y)`: Returns true if x and y are connected. Connections can be transitive, i.e. they don't need to be direct.

Useful for many purposes, e.g.:

- Percolation theory:
 - Computational chemistry.
- Implementation of other algorithms:
 - Kruskal's algorithm.



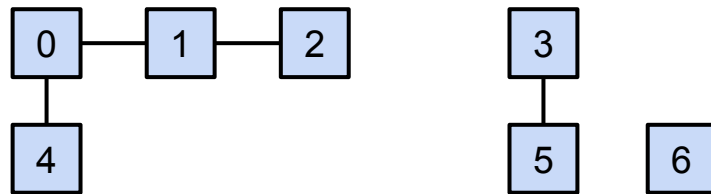
"Another major challenge is that there is nothing on either side of the Bering Strait to connect the bridge to."

Disjoint Sets on Integers

To keep things simple, we're going to:

- Force all items to be integers instead of arbitrary data (e.g. 8 instead of USA).
- Declare the number of items in advance, everything is disconnected at start.

```
ds = DisjointSets(7)
ds.connect(0, 1)
ds.connect(1, 2)
ds.connect(0, 4)
ds.connect(3, 5)
ds.isConnected(2, 4): true
ds.isConnected(3, 0): false
```

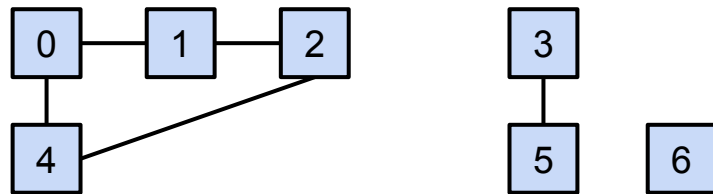


Disjoint Sets on Integers

To keep things simple, we're going to:

- Force all items to be integers instead of arbitrary data (e.g. 8 instead of USA).
- Declare the number of items in advance, everything is disconnected at start.

```
ds = DisjointSets(7)
ds.connect(0, 1)
ds.connect(1, 2)
ds.connect(0, 4)
ds.connect(3, 5)
ds.isConnected(2, 4): true
ds.isConnected(3, 0): false
ds.connect(4, 2)
```

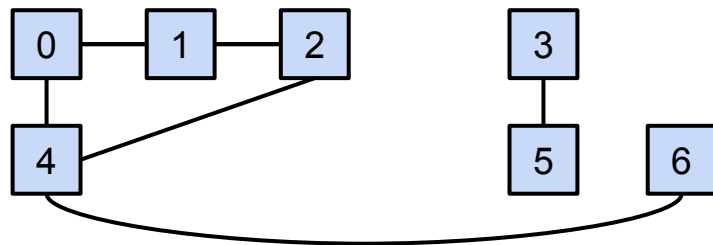


Disjoint Sets on Integers

To keep things simple, we're going to:

- Force all items to be integers instead of arbitrary data (e.g. 8 instead of USA).
- Declare the number of items in advance, everything is disconnected at start.

```
ds = DisjointSets(7)
ds.connect(0, 1)
ds.connect(1, 2)
ds.connect(0, 4)
ds.connect(3, 5)
ds.isConnected(2, 4): true
ds.isConnected(3, 0): false
ds.connect(4, 2)
ds.connect(4, 6)
```

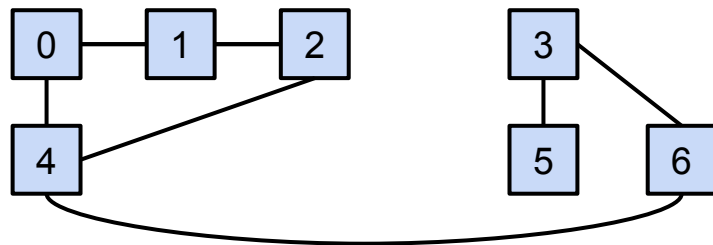


Disjoint Sets on Integers

To keep things simple, we're going to:

- Force all items to be integers instead of arbitrary data (e.g. 8 instead of USA).
- Declare the number of items in advance, everything is disconnected at start.

```
ds = DisjointSets(7)
ds.connect(0, 1)
ds.connect(1, 2)
ds.connect(0, 4)
ds.connect(3, 5)
ds.isConnected(2, 4): true
ds.isConnected(3, 0): false
ds.connect(4, 2)
ds.connect(4, 6)
ds.connect(3, 6)
```

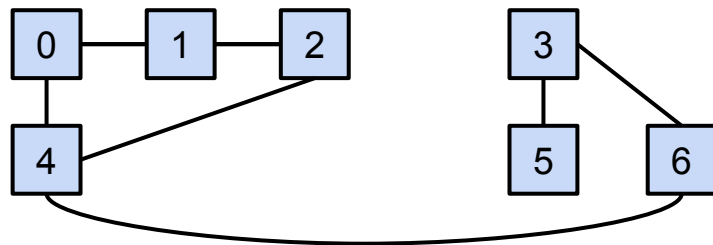


Disjoint Sets on Integers

To keep things simple, we're going to:

- Force all items to be integers instead of arbitrary data (e.g. 8 instead of USA).
- Declare the number of items in advance, everything is disconnected at start.

```
ds = DisjointSets(7)
ds.connect(0, 1)
ds.connect(1, 2)
ds.connect(0, 4)
ds.connect(3, 5)
ds.isConnected(2, 4): true
ds.isConnected(3, 0): false
ds.connect(4, 2)
ds.connect(4, 6)
ds.connect(3, 6)
ds.isConnected(3, 0): true
```



The Disjoint Sets Interface

```
public interface DisjointSets {  
    /** Connects two items p and q. */  
    void connect(int p, int q);  
  
    /** Checks to see if two items are connected. */  
    boolean isConnected(int p, int q);  
}
```

Goal: Design an efficient DisjointSets implementation.

- Number of elements N can be huge.
- Number of method calls M can be huge.
- Calls to methods may be interspersed (e.g. can't assume it's only connect operations followed by only isConnected operations).

connect(int p, int q)
isConnected(int p, int q)

Tracking Connected Components

Lecture 14, CS61B, Spring 2026

Big O and Big Theta Review

Introduction to Disjoint Sets

- Disjoint Sets API
- **Tracking Connected Components**

Implementations

- List of Sets
- Quick Find
- Quick Union
- Weighted Quick Union
- WQU with Path Compression

The Naive Approach

Naive approach:

- connect: Record every single connecting line in some data structure.
 - In Java, could be a list of pairs: `List<Integer[]>`



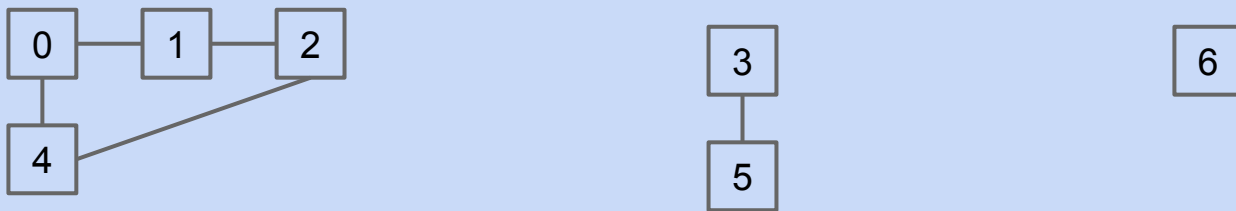
`[0, 1], [1, 2], [0, 4], [2, 4], [3, 5]`

`ds.isConnected(3, 0)`

The Naive Approach: Implementing isConnected

Naive approach:

- connect: Record every single connecting line in some data structure.
 - In Java, could be a list of pairs: `List<Integer[]>`



`[0, 1], [1, 2], [0, 4], [2, 4], [3, 5]`

`ds.isConnected(3, 0)`

How do we implement isConnected?

The Naive Approach

Naive approach:

- connect: Record every single connecting line in some data structure.
 - In Java, could be a list of pairs: `List<Integer[]>`



`[0, 1], [1, 2], [0, 4], [2, 4], [3, 5]`

`ds.isConnected(3, 0)` ← Hard! Do some computation over all of the pairs?

- `isConnected`: Difficult to determine if one thing can be reached from another!

A Better Approach: Connected Components

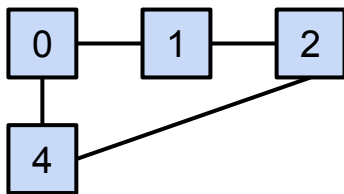
Rather than manually writing out every single connecting line, only record the sets that each item belongs to.

	$\{0\}, \{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}$
<code>connect(0, 1)</code>	$\{0, 1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}$
<code>connect(1, 2)</code>	$\{0, 1, 2\}, \{3\}, \{4\}, \{5\}, \{6\}$
<code>connect(0, 4)</code>	$\{0, 1, 2, 4\}, \{3\}, \{5\}, \{6\}$
<code>connect(3, 5)</code>	$\{0, 1, 2, 4\}, \{3, 5\}, \{6\}$
<code>isConnected(2, 4):</code>	<code>true</code>
<code>isConnected(3, 0):</code>	<code>false</code>
<code>connect(4, 2)</code>	$\{0, 1, 2, 4\}, \{3, 5\}, \{6\}$
<code>connect(4, 6)</code>	$\{0, 1, 2, 4, 6\}, \{3, 5\}$
<code>connect(3, 6)</code>	$\{0, 1, 2, 3, 4, 5, 6\}$
<code>isConnected(3, 0):</code>	<code>true</code>

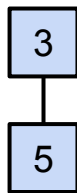
A Better Approach: Connected Components

For each item, its **connected component** is the set of all items that are connected to that item.

- Naive approach: Record every single connecting line somehow.
- Better approach: Model connectedness in terms of sets.
 - How things are connected isn't something we need to know.
 - Only need to keep track of which connected component each item belongs to.



{ 0, 1, 2, 4 }



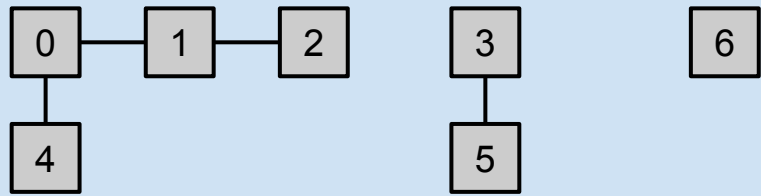
{ 3, 5 }



{ 6 }

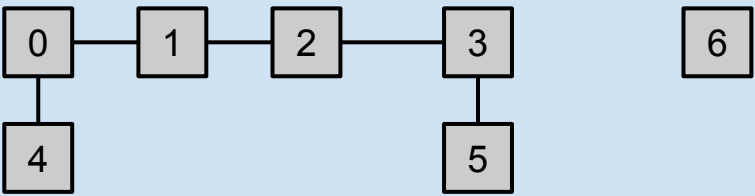
Challenge: Pick Data Structures to Support Tracking of Sets: yellkey.com/disease

Before connect(2, 3) operation:



{ 0, 1, 2, 4 }, {3, 5}, {6}

After connect(2, 3) operation:



{ 0, 1, 2, 4, 3, 5}, {6}

Attendance:



List of Sets

Lecture 14, CS61B, Spring 2026

Big O and Big Theta Review

Introduction to Disjoint Sets

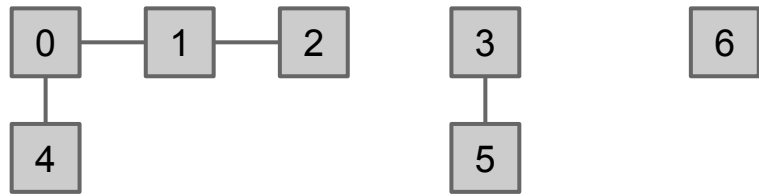
- Disjoint Sets API
- Tracking Connected Components

Implementations

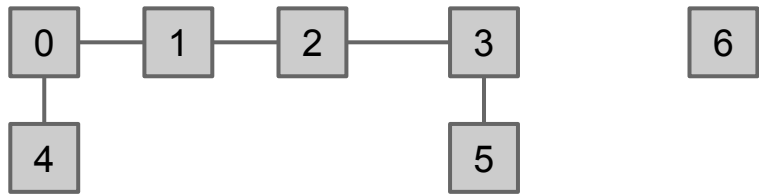
- **List of Sets**
- Quick Find
- Quick Union
- Weighted Quick Union
- WQU with Path Compression

Challenge: Pick Data Structures to Support Tracking of Sets

Before connect(2, 3) operation:



After connect(2, 3) operation:



{ 0, 1, 2, 4 }, {3, 5}, {6}

{ 0, 1, 2, 4, 3, 5}, {6}

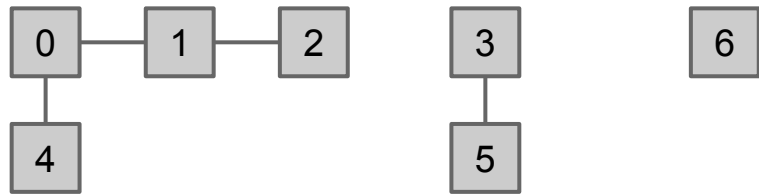
Idea #1: List of sets of integers, e.g. [{0, 1, 2, 4}, {3, 5}, {6}]

- In Java: List<Set<Integer>>

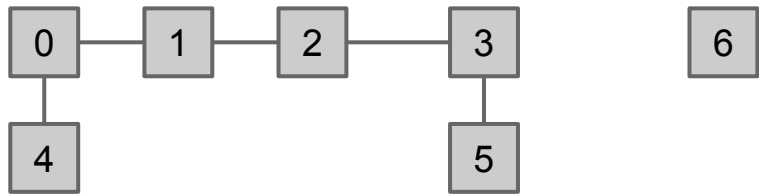
Implementation	constructor	connect	isConnected
ListOfSetsDS			

Challenge: Pick Data Structures to Support Tracking of Sets

Before connect(2, 3) operation:



After connect(2, 3) operation:



{ 0, 1, 2, 4 }, {3, 5}, {6}

{ 0, 1, 2, 4, 3, 5}, {6}

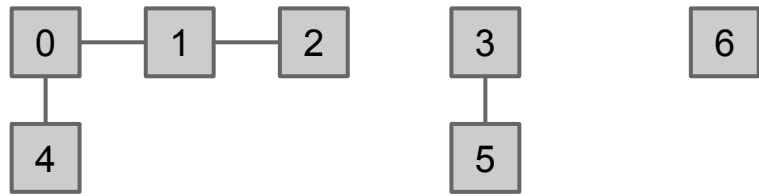
Idea #1: List of sets of integers, e.g. [{0, 1, 2, 4}, {3, 5}, {6}]

- In Java: List<Set<Integer>>

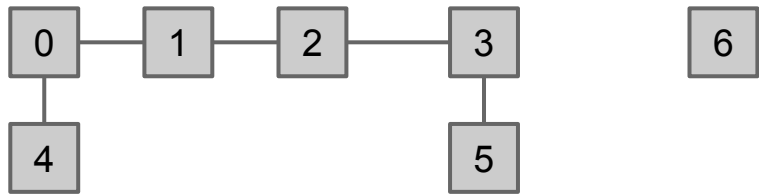
Implementation	constructor	connect	isConnected
ListOfSetsDS	$\Theta(N)$ Make N sets		

Challenge: Pick Data Structures to Support Tracking of Sets

Before connect(2, 3) operation:



After connect(2, 3) operation:



{ 0, 1, 2, 4 }, {3, 5}, {6}

{ 0, 1, 2, 4, 3, 5}, {6}

Idea #1: List of sets of integers, e.g. [{0, 1, 2, 4}, {3, 5}, {6}]

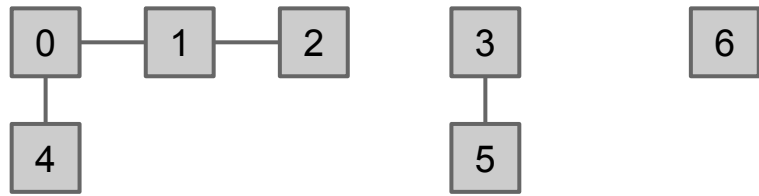
- In Java: List<Set<Integer>>

Worst case is $\Theta(N)$, but other cases may be better. We'll say $O(N)$ since O means "less than or equal".

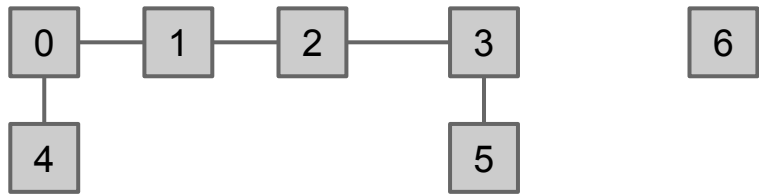
Implementation	constructor	connect	isConnected
ListOfSetsDS	$\Theta(N)$ Make N sets	$O(N)$ Find each element, Combine 2 sets	

Challenge: Pick Data Structures to Support Tracking of Sets

Before connect(2, 3) operation:



After connect(2, 3) operation:



{ 0, 1, 2, 4 }, {3, 5}, {6}

{ 0, 1, 2, 4, 3, 5 }, {6}

Idea #1: List of sets of integers, e.g. [{0, 1, 2, 4}, {3, 5}, {6}]

- In Java: List<Set<Integer>>

Worst case is $\Theta(N)$, but other cases may be better. We'll say $O(N)$ since O means "less than or equal".

Implementation	constructor	connect	isConnected
ListOfSetsDS	$\Theta(N)$ Make N sets	$O(N)$ Find each element, Combine 2 sets	$O(N)$ Check up to N sets

Idea #1: ListOfSetsDS

List of sets of integers, e.g. [{0, 1, 2, 4}, {3, 5}, {6}]

- In Java: `List<Set<Integer>>`

Worst case is $\Theta(N)$, but other cases may be better. We'll say $O(N)$ since O means "less than or equal".

Implementation	constructor	connect	isConnected
ListOfSetsDS	$\Theta(N)$ Make N sets	$O(N)$ Find each element, Combine 2 sets	$O(N)$ Check up to N sets

- Very intuitive idea, but actually **terrible!**
- Requires iterating through all the sets to find anything. Complicated and slow!
 - Worst case: If nothing is connected, then need to iterate through all N sets to find anything
- Important point: Data structure choice - List of Sets - doomed us!

QuickFind

Lecture 14, CS61B, Spring 2026

Big O and Big Theta Review

Introduction to Disjoint Sets

- Disjoint Sets API
- Tracking Connected
- Components

Implementations

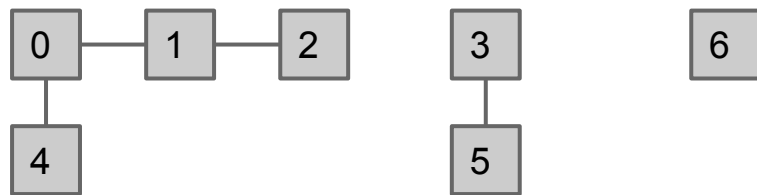
- List of Sets
- **Quick Find**
- Quick Union
- Weighted Quick Union
- WQU with Path Compression

Next Approach: Array of Integers

Idea #2: array of integers where i th entry gives set number (a.k.a. “id”) of item i .

- `connect(p, q)`: Change entries that equal `id[p]` to `id[q]`

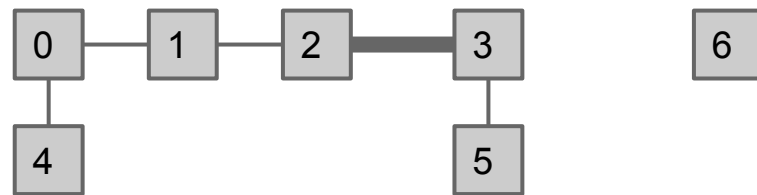
Before `connect(2, 3)` operation:



{ 0, 1, 2, 4 }, {3, 5}, {6}

int[] id	4	4	4	5	4	5	6
	0	1	2	3	4	5	6

After `connect(2, 3)` operation:



{ 0, 1, 2, 4, 3, 5 }, {6}

int[] id	5	5	5	5	5	5	6
	0	1	2	3	4	5	6

`isConnected(3, 6): id(3) == id(6)` (**false**)

```
public class QuickFindDS implements DisjointSets {  
    private int[] id;
```

```
    public boolean isConnected(int p, int q) {  
        return id[p] == id[q];  
    }
```

Very fast: Two array accesses: $\Theta(1)$

```
    public void connect(int p, int q) {  
        int pid = id[p];  
        int qid = id[q];  
        for (int i = 0; i < id.length; i++) {  
            if (id[i] == pid) {  
                id[i] = qid;  
            }  
        }  
    }...
```

Relatively slow: $N+2$ to $2N+2$ array accesses: $\Theta(N)$

```
    public QuickFindDS(int N) {  
        id = new int[N];  
        for (int i = 0; i < N; i++)  
            id[i] = -1;  
    }
```


Implementation	constructor	connect	isConnected
ListOfSetsDS	$\Theta(N)$ Make N sets	$O(N)$ Find each element, Combine 2 sets	$O(N)$ Check up to N sets
QuickFindDS	$\Theta(N)$ Initialize size-N array	$\Theta(N)$ Look through entire array, update IDs	$\Theta(1)$ Two array accesses

QuickFindDS is too slow for practical use: Connecting two items takes N time.

- Instead, let's try something more radical.

Quick Union

Lecture 14, CS61B, Spring 2026

Big O and Big Theta Review

Introduction to Disjoint Sets

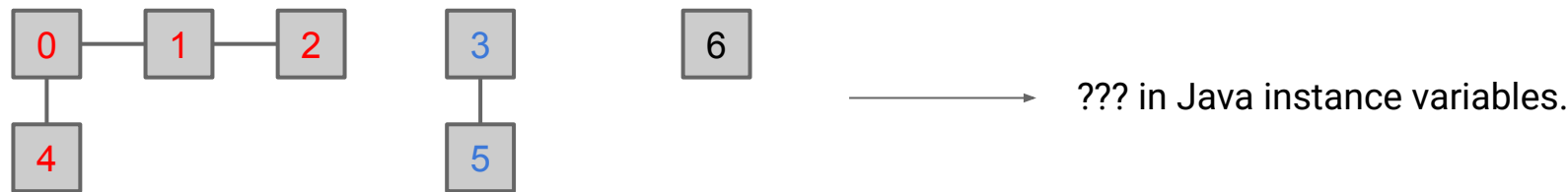
- Disjoint Sets API
- Tracking Connected Components

Implementations

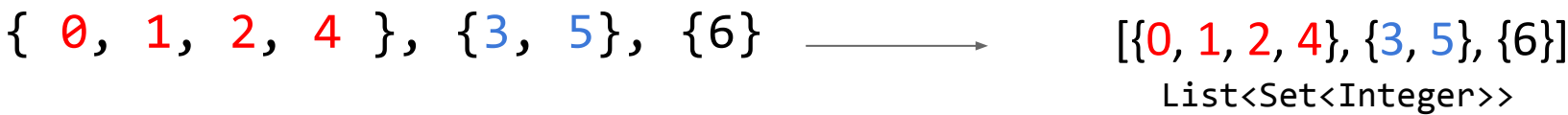
- List of Sets
- Quick Find
- **Quick Union**
- Weighted Quick Union
- WQU with Path Compression

Improving the Connect Operation

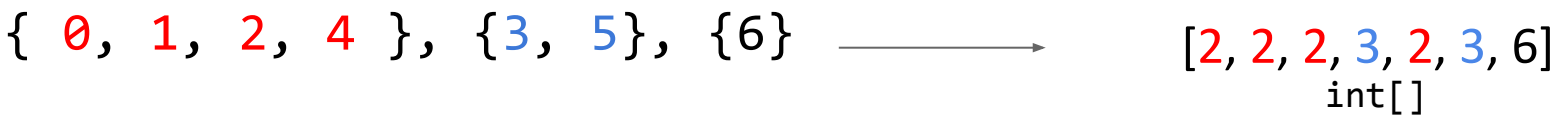
Approach zero: Represent everything as boxes and lines. Overly complicated.



ListOfSets: Represent everything as connected components. Represented connected components as list of sets of integers.



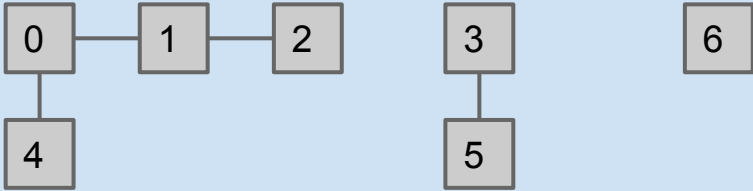
QuickFind: Represent everything as connected components. Represented connected components as a list of integers, where value = id.



Improving the Connect Operation

Hard question: How can we make `connect()` fast? Can we avoid looking through the entire array?

QuickFind version:

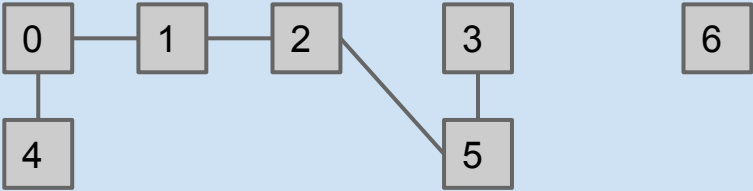


{ 0, 1, 2, 4 }, {3, 5}, {6}

id

0	0	0	3	0	3	6
0	1	2	3	4	5	6

`connect(5, 2)`



{ 0, 1, 2, 4, 3, 5 }, {6}

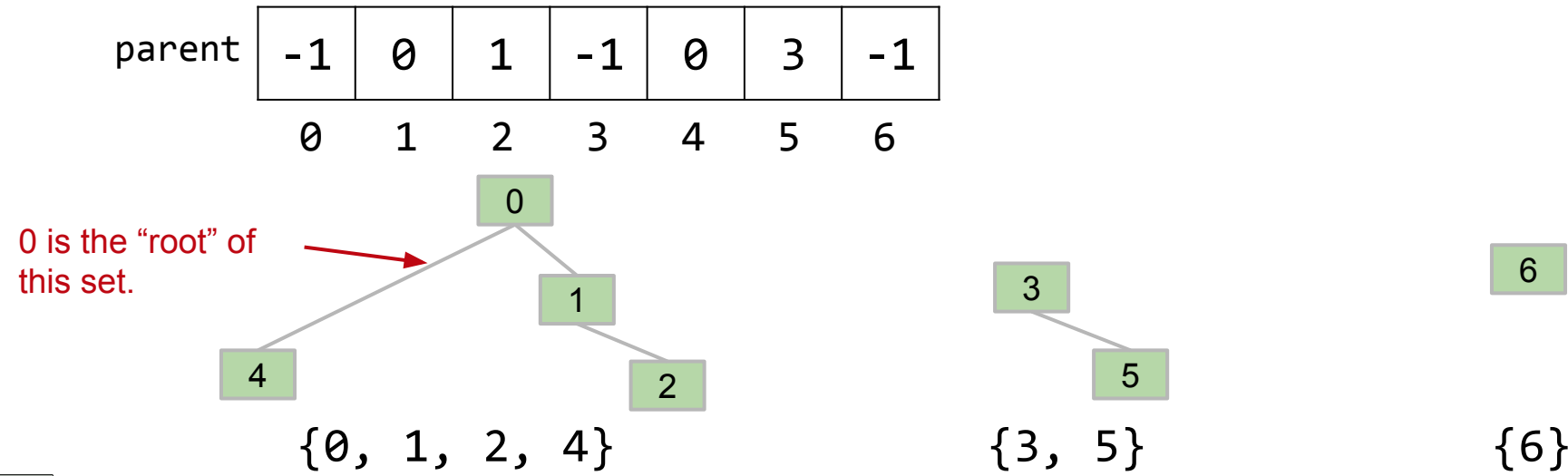
id

3	3	3	3	3	3	6
0	1	2	3	4	5	6

Improving the Connect Operation

Hard question: How can we make `connect()` fast? Can we avoid looking through the entire array?

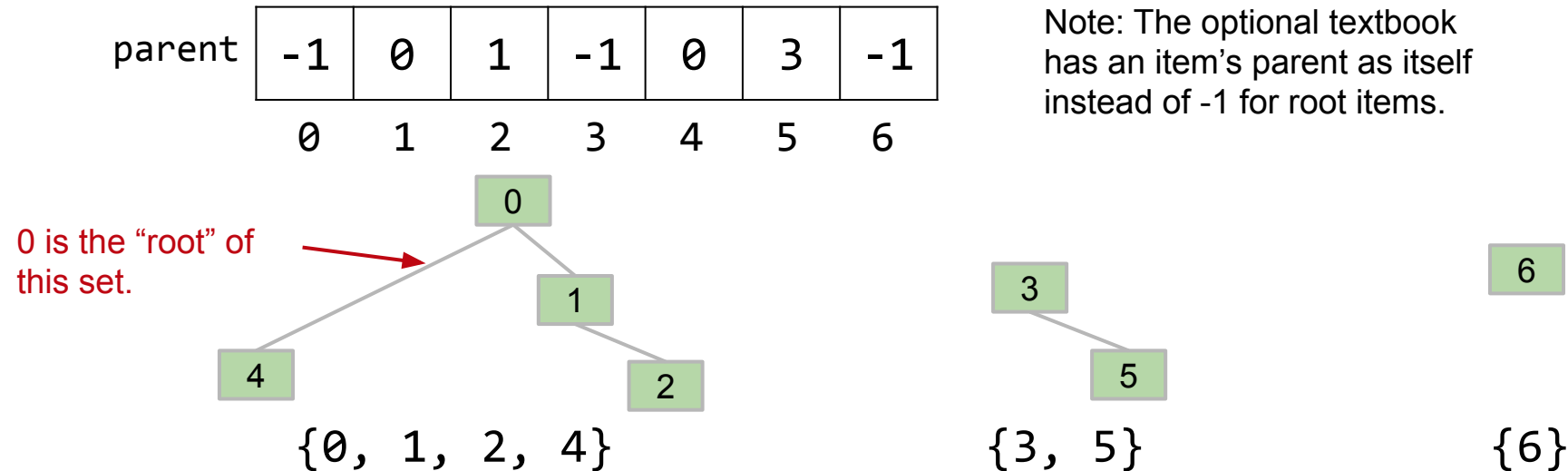
- Idea: Assign each item a parent (instead of an id). Results in a tree-like shape.
 - An innocuous sounding, seemingly arbitrary solution.
 - Unlocks a pretty amazing universe of math that we won't discuss.



Improving the Connect Operation

Hard question: How can we make `connect()` fast? Can we avoid looking through the entire array?

- Idea: Assign each item a parent (instead of an id). Results in a tree-like shape.
 - An innocuous sounding, seemingly arbitrary solution.
 - Unlocks a pretty amazing universe of math that we won't discuss.

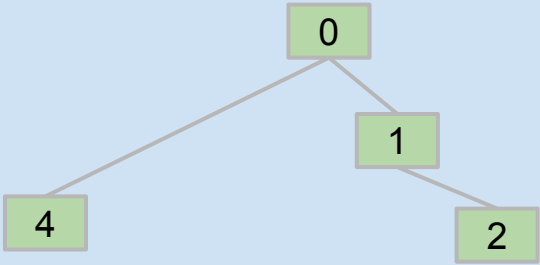


connect(5, 2)

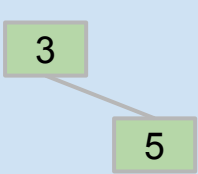
- How should we change the parent list to handle this connect operation?
 - If you're not sure where to start, consider: why can't we just set id[5] to 2?

parent

-1	0	1	0	0	3	-1
0	1	2	3	4	5	6



{0, 1, 2, 4}



{3, 5}

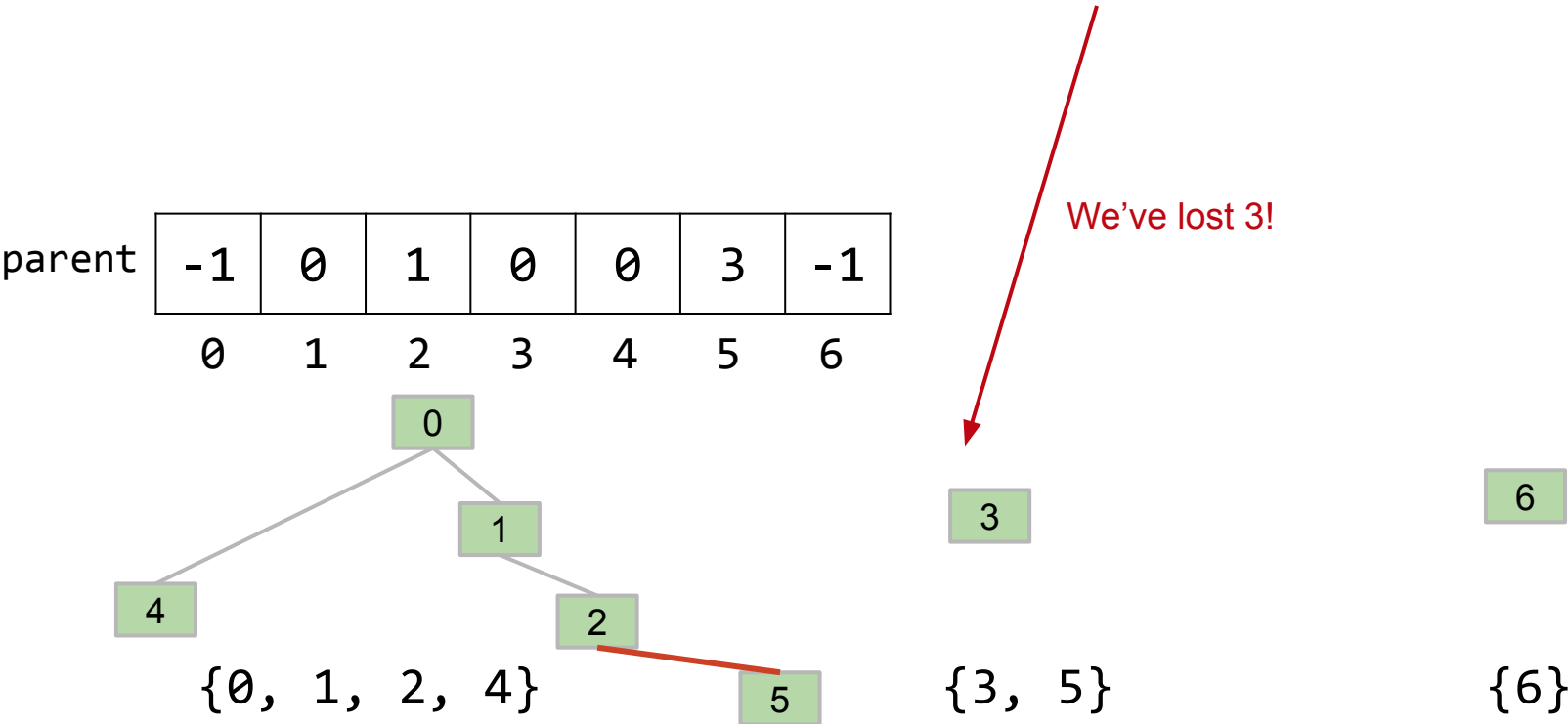


{6}

Improving the Connect Operation

connect(5, 2)

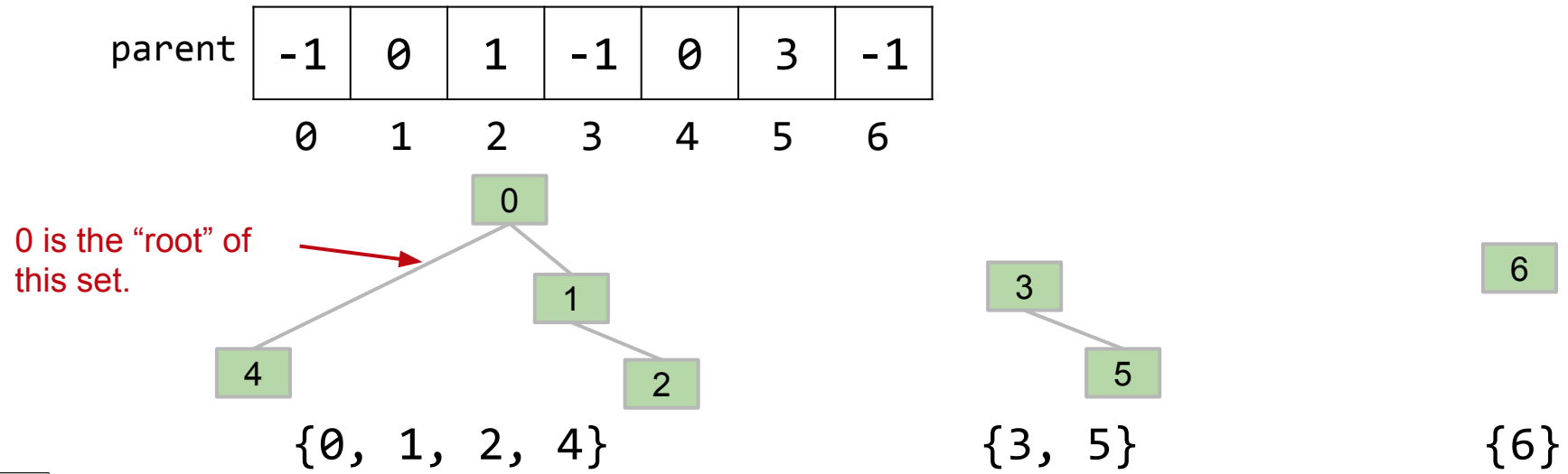
- How should we change the parent list to handle this connect operation?
 - If you're not sure where to start, consider: why can't we just set id[5] to 2?



Improving the Connect Operation

connect(5, 2)

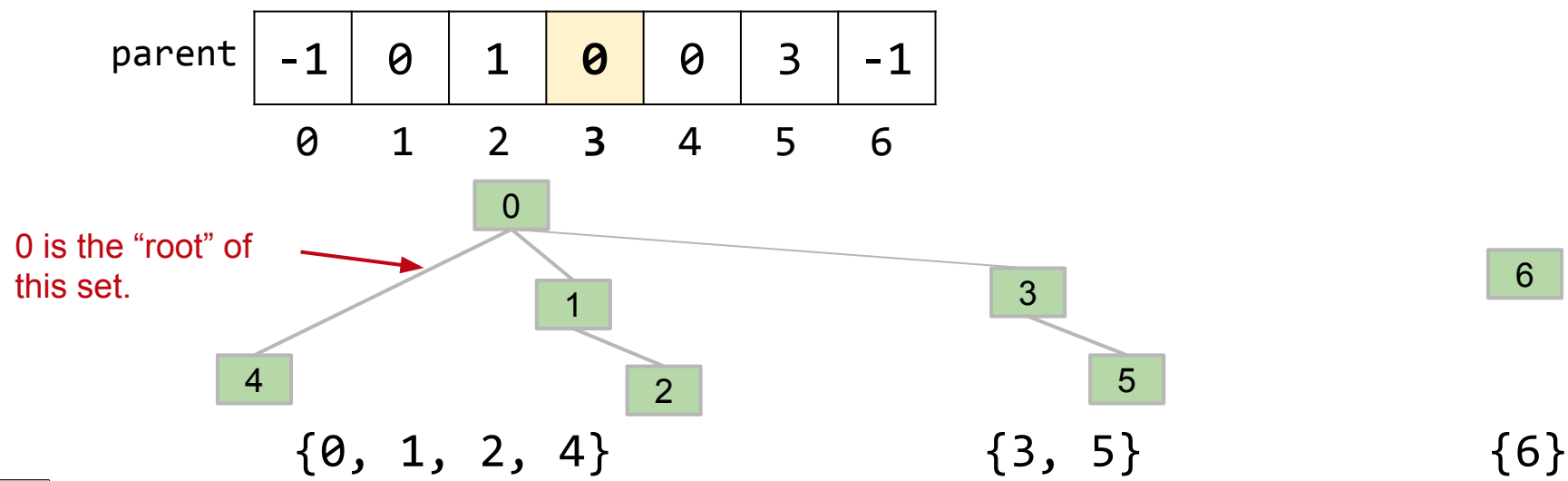
- Find root(5). // returns 3
- Find root(2). // returns 0
- Set root(5)'s value equal to root(2).



Improving the Connect Operation

connect(5, 2)

- Find root(5). // returns 3
- Find root(2). // returns 0
- Set parent[root(5)] = root(2).

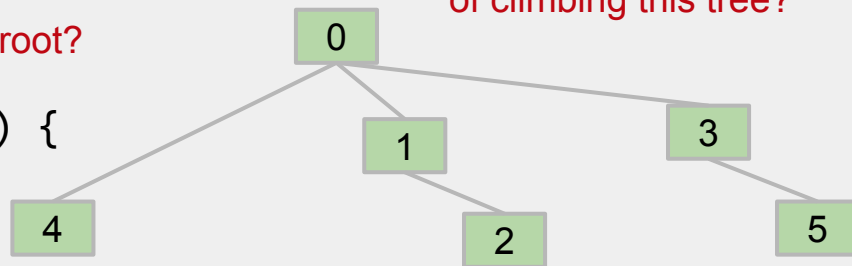


```
public class QuickUnionDS implements DisjointSets {  
    private int[] parent;  
    public QuickUnionDS(int N) {  
        parent = new int[N];  
        for (int i = 0; i < N; i++)  
            { parent[i] = -1; }  
    }  
}
```

```
public boolean isConnected(int p, int q) {  
    return root(p) == root(q);  
}
```

What's the runtime of root?

```
public void connect(int p, int q) {  
    int i = root(p);  
    int j = root(q);  
    parent[i] = j;  
}
```



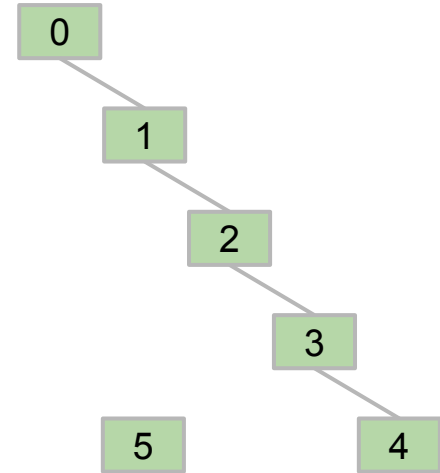
What's the runtime of climbing this tree?

Optional textbook uses find instead of root

The Worst Case

If we always connect the first item's tree below the second item's tree, we can end up with a tree of height M after M operations:

- `connect(4, 3)`
- `connect(3, 2)`
- `connect(2, 1)`
- `connect(1, 0)`



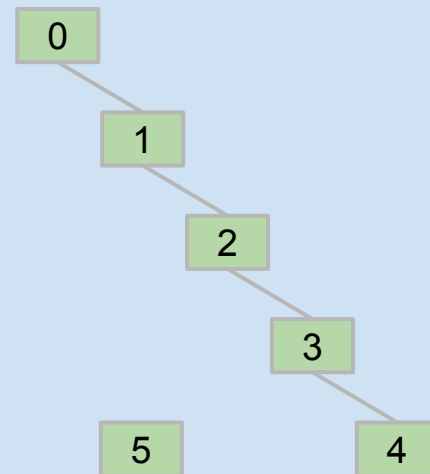
The Worst Case

If we always connect the first item's tree below the second item's tree, we can end up with a tree of height M after M operations:

- `connect(4, 3)`
- `connect(3, 2)`
- `connect(2, 1)`
- `connect(1, 0)`

For N items, what's the worst case runtime...

- For `connect(p, q)`?
- For `isConnected(p, q)`?



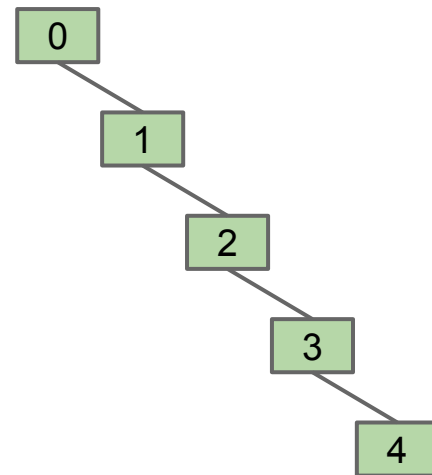
The Worst Case

If we always connect the first item's tree below the second item's tree, we can end up with a tree of height M after M operations:

- `connect(4, 3)`
- `connect(3, 2)`
- `connect(2, 1)`
- `connect(1, 0)`

For N items, what's the worst case runtime...

- For `connect(p, q)`? $\Theta(N)$
- For `isConnected(p, q)`? $\Theta(N)$



```
public class QuickUnionDS implements DisjointSets {  
    private int[] parent;  
    public QuickUnionDS(int N) {  
        parent = new int[N];  
        for (int i = 0; i < N; i++)  
            { parent[i] = -1; }  
    }
```

For N items, this means a worst case runtime of $\Theta(N)$.



```
    public boolean isConnected(int p, int q) {  
        return root(p) == root(q);  
    }
```

```
    public void connect(int p, int q) {  
        int i = root(p);  
        int j = root(q);  
        parent[i] = j;  
    }  
}
```

```
private int root(int p) {  
    int r = p;  
    while (parent[r] >= 0) {  
        r = parent[r];  
    }  
    return r;  
}
```

Optional textbook uses find instead of root

```
public class QuickUnionDS implements DisjointSets {  
    private int[] parent;  
    public QuickUnionDS(int N) {  
        parent = new int[N];  
        for (int i = 0; i < N; i++)  
            { parent[i] = -1; }  
    }
```

For N items, this means a worst case runtime of $\Theta(N)$.

```
private int find(int p) {  
    int r = p;  
    while (parent[r] >= 0)  
        { r = parent[r]; }  
    return r;  
}
```

Here the find operation is the same as the “root(x)” idea we had in earlier slides.

```
public boolean isConnected(int p, int q) {  
    return find(p) == find(q);  
}  
  
public void connect(int p, int q) {  
    int i = find(p);  
    int j = find(q);  
    parent[i] = j;  
}
```


Performance Summary

Implementation	constructor	connect	isConnected
ListOfSetsDS	$\Theta(N)$ Make N sets	$O(N)$ Find each element, Combine 2 sets	$O(N)$ Check up to N sets
QuickFindDS	$\Theta(N)$ Initialize size-N array	$\Theta(N)$ Look through entire array, update IDs	$\Theta(1)$ Two array accesses
QuickUnionDS	$\Theta(N)$ Initialize size-N array	$O(N)$ Climb tree to find root	$O(N)$ Climb 2 tree to compare roots

Using O because
runtime can be
between constant
and linear.

QuickUnion defect: Trees can get tall. Connecting takes $\Theta(N)$ time.

- Results in potentially even worse performance than QuickFind if tree is imbalanced.
- Observation: Things would be fine if we just kept our tree balanced.

Weighted Quick Union

Lecture 14, CS61B, Spring 2026

Big O and Big Theta Review

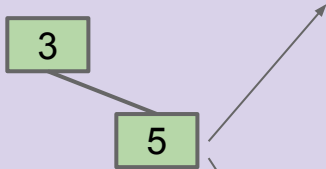
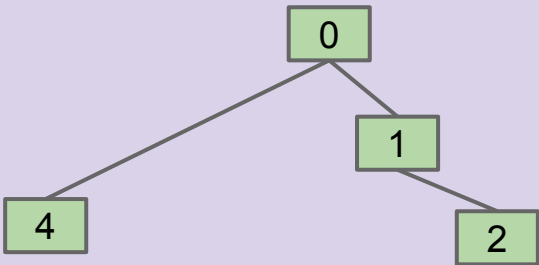
Introduction to Disjoint Sets

- Disjoint Sets API
- Tracking Connected Components

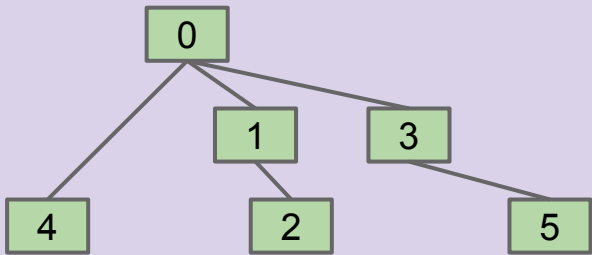
Implementations

- List of Sets
- Quick Find
- Quick Union
- **Weighted Quick Union**
- WQU with Path Compression

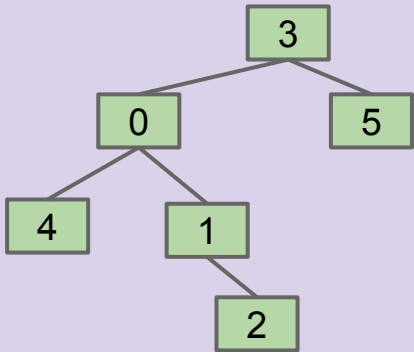
connect(2, 5)



A. Make 5's root into a child of 2's root.



B. Make 2's root into a child of 5's root.

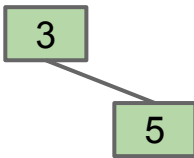
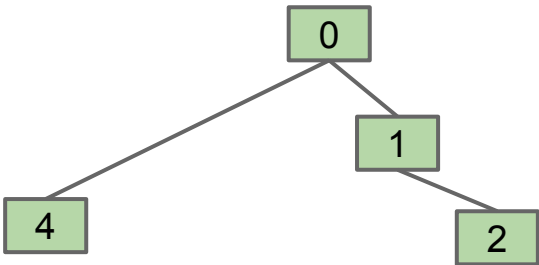


Which is the better choice?



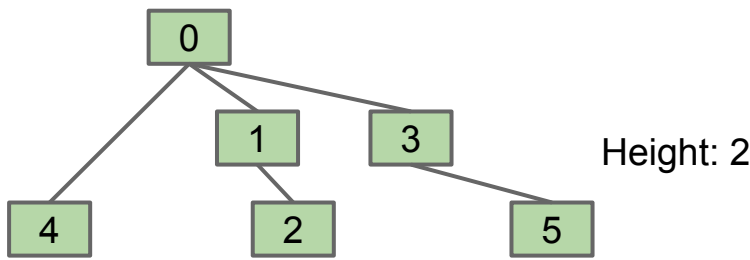
A Choice of Two Roots

connect(2, 5)

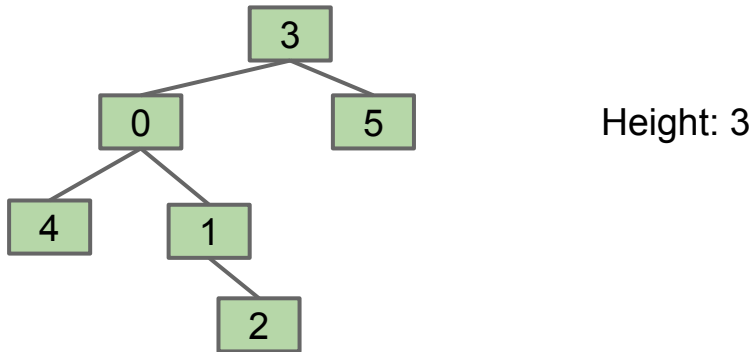


Which is the better choice?

A. Make 5's root into a child of 2's root.



B. Make 2's root into a child of 5's root.



Possible Approach

One possible approach is to keep track of the height of every tree.

- Link up shorter tree below the larger tree.
- In case of a tie, break tie arbitrarily.

Unfortunately, tracking tree height becomes difficult with upcoming optimizations we'll make.

Interestingly, tracking the tree's "size", a.k.a. "weight" works just as well asymptotically.

- Size and weight both mean the total number of items in that tree.

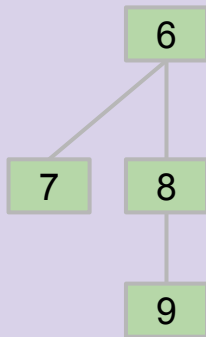
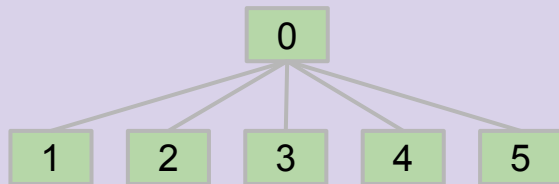


Modify quick-union to avoid tall trees.

- Track tree size (**number** of elements).
- New rule: Always link root of **smaller** tree **to larger** tree.

New rule: If we call `connect(3, 8)`, which entry (or entries) of `parent[]` changes?

- A. `parent[3]`
- B. `parent[0]`
- C. `parent[8]`
- D. `parent[6]`



parent	-1	0	0	0	0	0	-1	6	6	8
	0	1	2	3	4	5	6	7	8	9

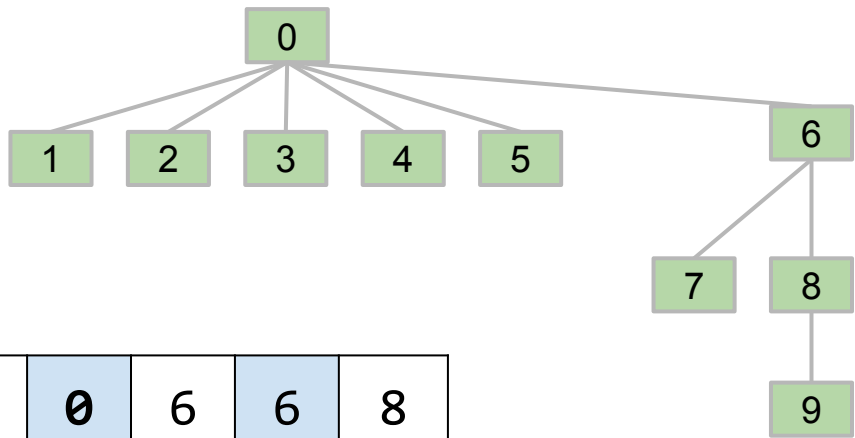
Improvement #1: Weighted QuickUnion

Modify quick-union to avoid tall trees.

- Track tree size (**number** of elements).
- New rule: Always link root of **smaller** tree **to larger** tree.

New rule: If we call `connect(3, 8)`, which entry (or entries) of `parent[]` changes?

- A. `parent[3]`
- B. `parent[0]`
- C. `parent[8]`
- D. **`parent[6]`**



parent	-1	0	0	0	0	0	6	6	8
	0	1	2	3	4	5	6	7	8

Minimal changes needed:

- Use `parent[]` array as before.
- `isConnected(int p, int q)` requires no changes.
- `connect(int p, int q)` needs to somehow keep track of sizes.
 - Two common approaches:
 - Replace -1 with -weight for roots

parent	-2	-1	-1	-1	-1	0	-4	6	6	8
	0	1	2	3	4	5	6	7	8	9

- Create a separate size array

size	2	1	1	1	1	1	4	1	2	1
	0	1	2	3	4	5	6	7	8	9

Weighted Quick Union Performance

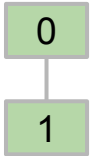
Let's consider the worst case where the tree height grows as fast as possible.

N	Height
1	0

0

Weighted Quick Union Performance

Let's consider the worst case where the tree height grows as fast as possible.

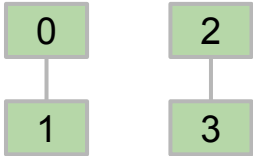


N	Height
1	0
2	1

Weighted Quick Union Performance

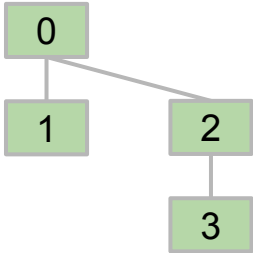
Let's consider the worst case where the tree height grows as fast as possible.

N	Height
1	0
2	1



Weighted Quick Union Performance

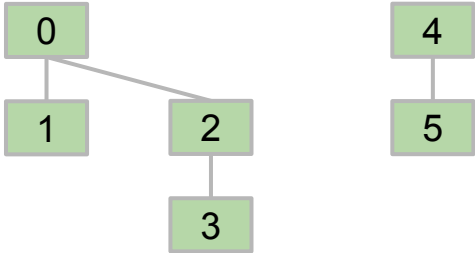
Let's consider the worst case where the tree height grows as fast as possible.



N	Height
1	0
2	1
4	2

Weighted Quick Union Performance

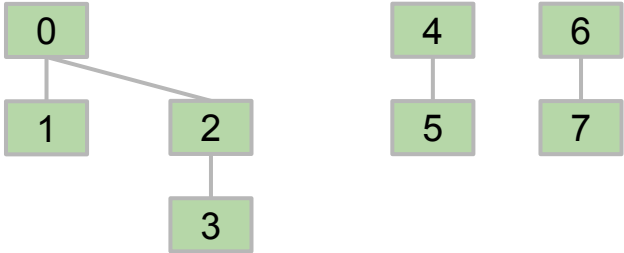
Let's consider the worst case where the tree height grows as fast as possible.



N	Height
1	0
2	1
4	2

Weighted Quick Union Performance

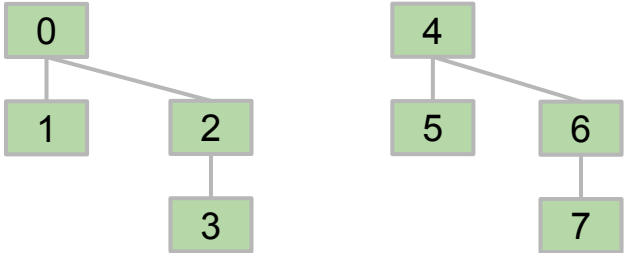
Let's consider the worst case where the tree height grows as fast as possible.



N	Height
1	0
2	1
4	2

Weighted Quick Union Performance

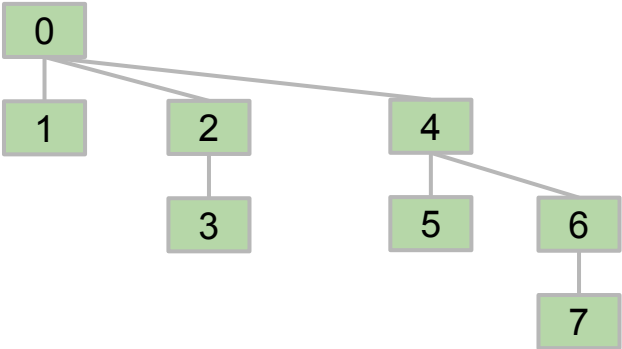
Let's consider the worst case where the tree height grows as fast as possible.



N	Height
1	0
2	1
4	2

Weighted Quick Union Performance

Let's consider the worst case where the tree height grows as fast as possible.

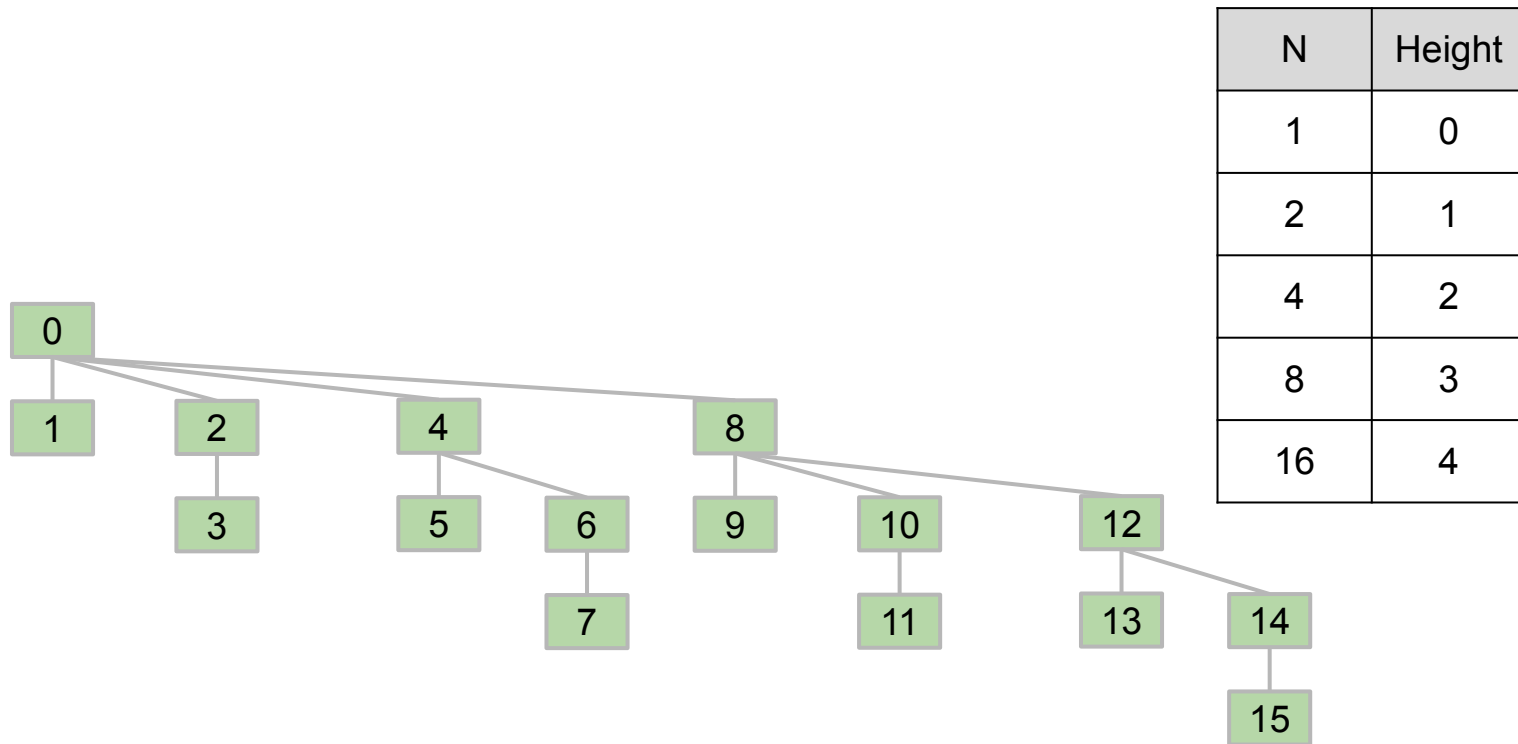


N	Height
1	0
2	1
4	2
8	3

Weighted Quick Union Performance

Let's consider the worst case where the tree height grows as fast as possible.

- Worst case tree height is $\Theta(\log N)$.



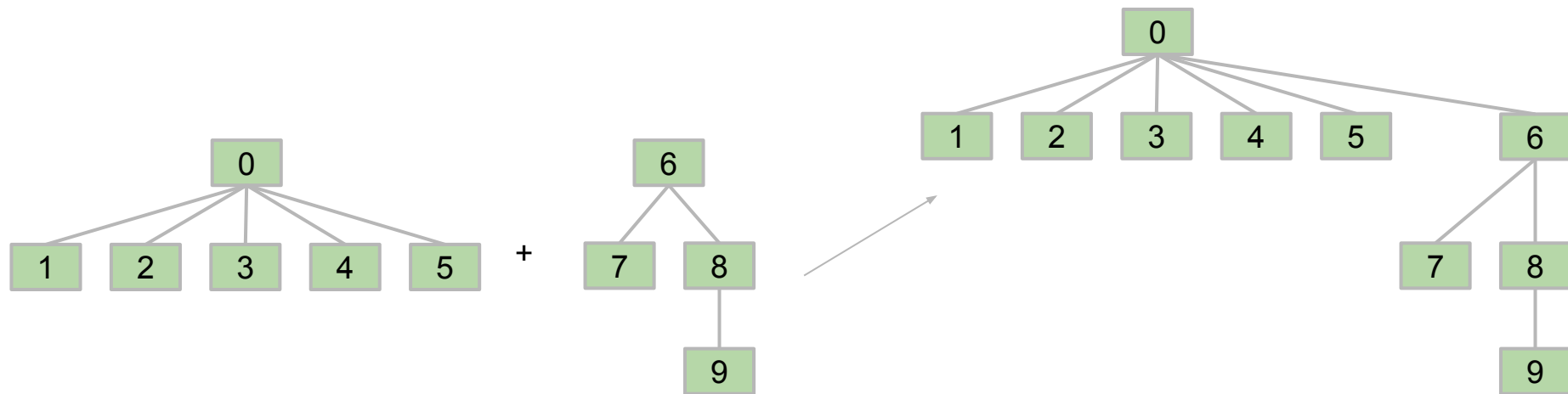
Performance Summary

Implementation	constructor	connect	isConnected
ListOfSetsDS	$\Theta(N)$ Make N sets	$O(N)$ Find each element, Combine 2 sets	$O(N)$ Check up to N sets
QuickFindDS	$\Theta(N)$ Initialize size-N array	$\Theta(N)$ Look through entire array, update IDs	$\Theta(1)$ Two array accesses
QuickUnionDS	$\Theta(N)$ Initialize size-N array	$O(N)$ Climb tree to find root	$O(N)$ Climb 2 tree to compare roots
WeightedQuick UnionDS	$\Theta(N)$ Initialize size-N array	$O(\log N)$ Climb tree to find root	$O(\log N)$ Climb 2 tree to compare roots

Why Weights Instead of Heights?

We used the number of items in a tree to decide upon the root.

- Why not use the height of the tree?
 - Worst case performance for HeightedQuickUnionDS is asymptotically the same! Both are $\Theta(\log(N))$.
 - Resulting code is more complicated with no performance gain.



WQU with Path Compression

Lecture 14, CS61B, Spring 2026

Big O and Big Theta Review

Introduction to Disjoint Sets

- Disjoint Sets API
- Tracking Connected Components

Implementations

- List of Sets
- Quick Find
- Quick Union
- Weighted Quick Union
- **WQU with Path Compression**

What We've Achieved

Implementation	constructor	connect	isConnected
ListOfSetsDS	$\Theta(N)$ Make N sets	$O(N)$ Find each element, Combine 2 sets	$O(N)$ Check up to N sets
WeightedQuick UnionDS	$\Theta(N)$ Initialize size-N array	$O(\log N)$ Climb tree to find root	$O(\log N)$ Climb 2 tree to compare roots

Performing M operations on a DisjointSet object with N elements:

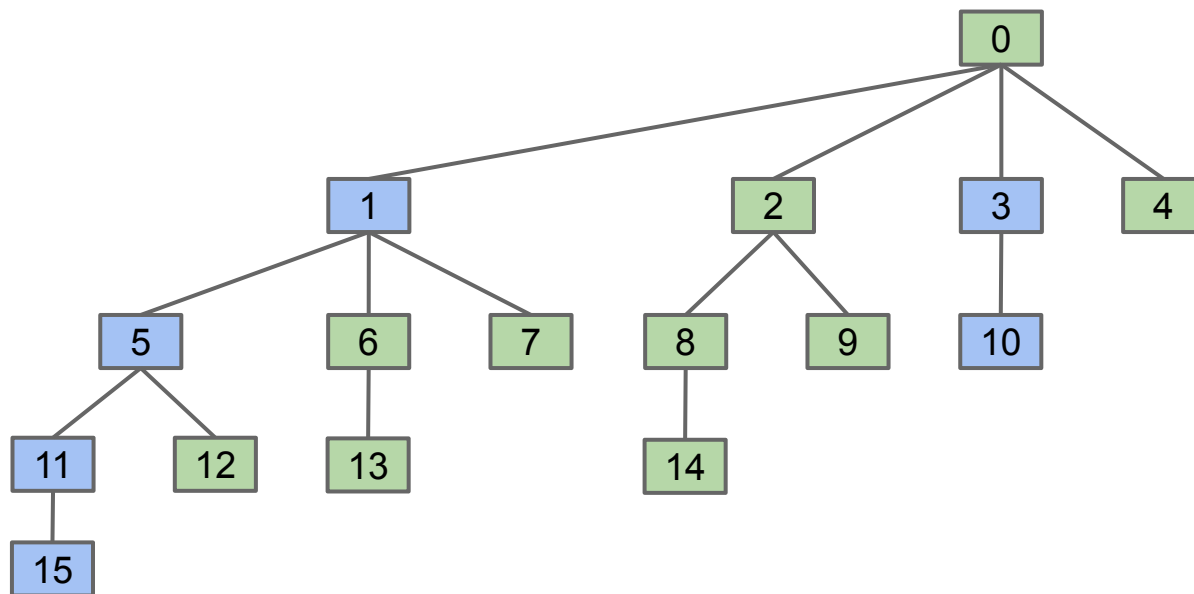
- For our naive implementation, runtime is $O(N + MN)$ or just $O(MN)$.
- For our best implementation, runtime is $O(N + M \log N)$.
- For $N = 10^9$ and $M = 10^9$, difference is 30 years vs. 6 seconds.
 - Key point: Good data structure unlocks solutions to problems that could otherwise not be solved!
- Good enough for all practical uses, but could we theoretically do better?

170 Spoiler: Path Compression: A Clever Idea

Below is the topology of the worst case if we use WeightedQuickUnion.

Clever idea: When we do `isConnected(15, 10)`, tie all nodes seen to the root.

- Additional cost is insignificant (same order of growth).

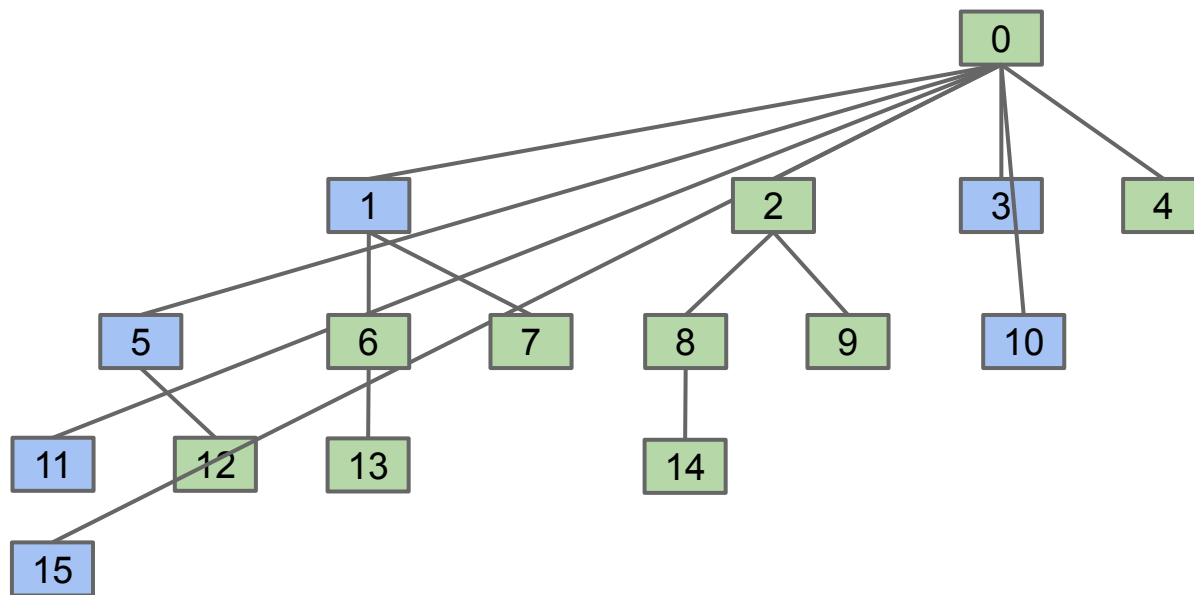


170 Spoiler: Path Compression: A Clever Idea

Below is the topology of the worst case if we use WeightedQuickUnion.

Clever idea: When we do `isConnected(15, 10)`, tie all nodes seen to the root.

- Additional cost is insignificant (same order of growth).

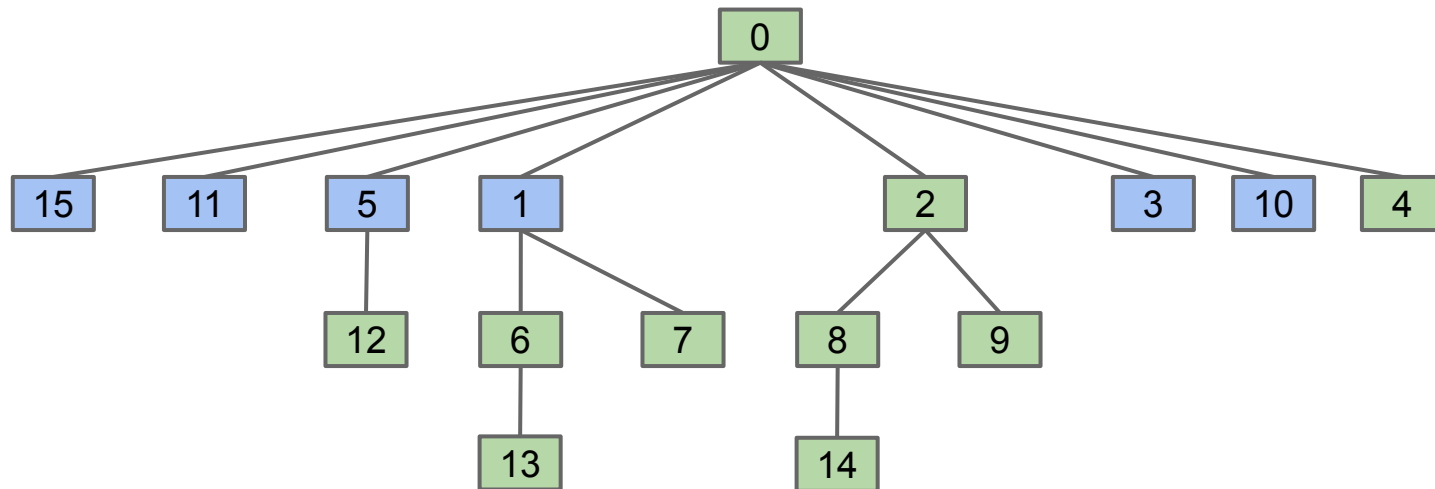


170 Spoiler: Path Compression: A Clever Idea

Below is the topology of the worst case if we use WeightedQuickUnion.

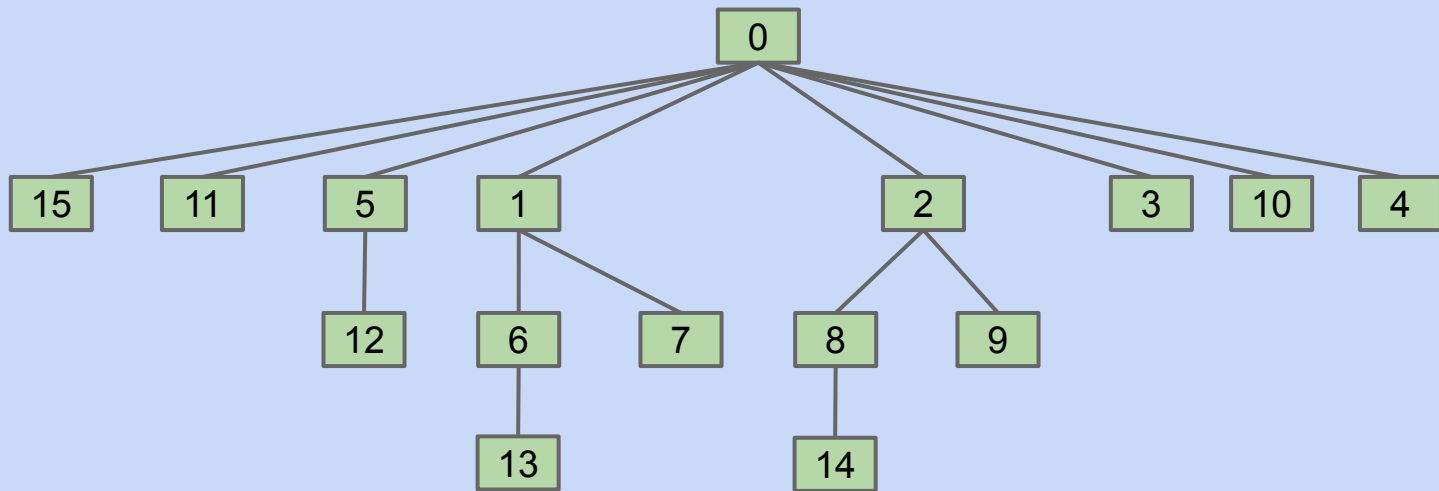
Clever idea: When we do `isConnected(15, 10)`, tie all nodes seen to the root.

- Additional cost is insignificant (same order of growth).



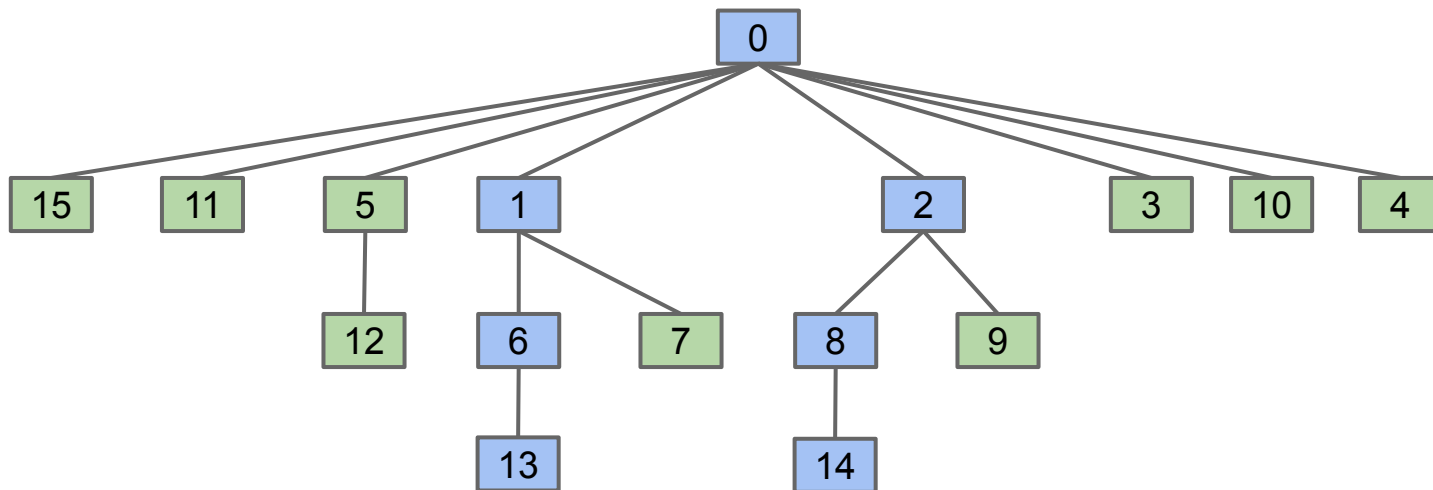
Path Compression: Another Clever Idea

Draw the tree after we call `isConnected(14, 13)`.



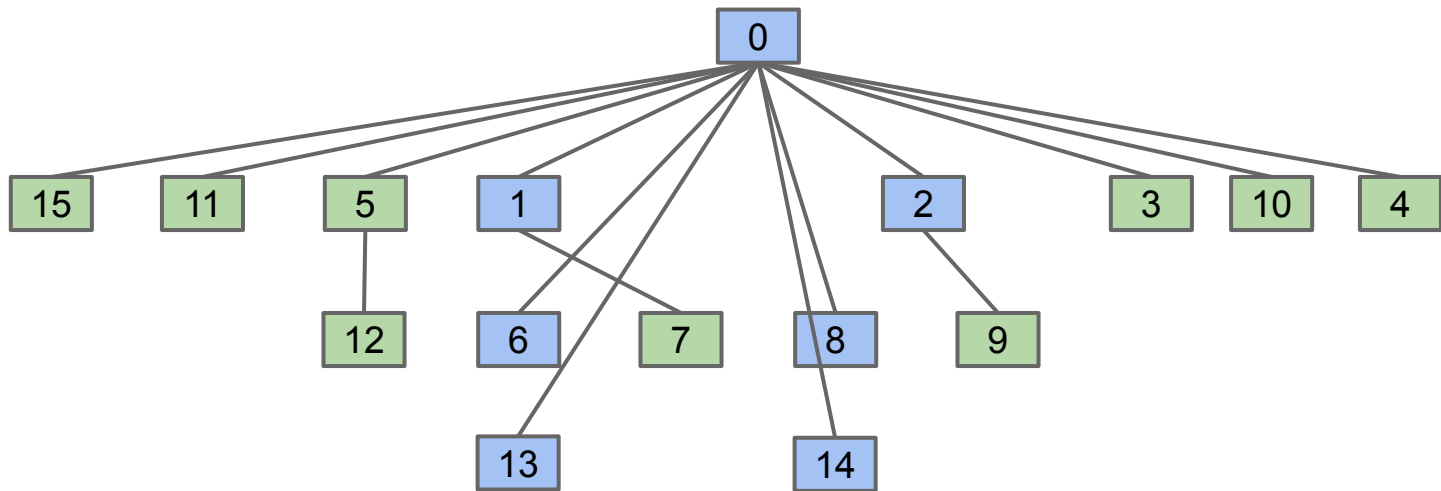
Path Compression: Another Clever Idea

Draw the tree after we call `isConnected(14, 13)`.



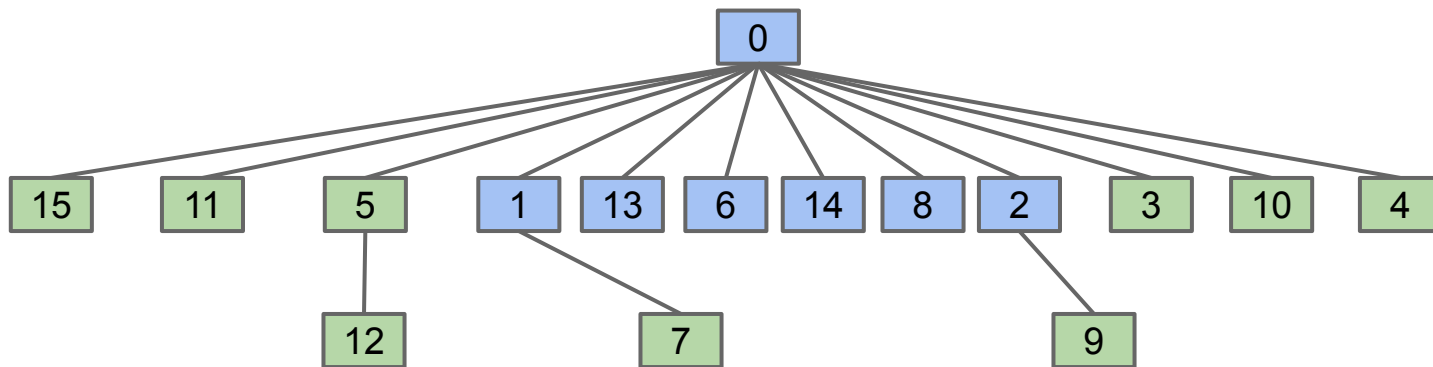
Path Compression: Another Clever Idea

Draw the tree after we call `isConnected(14, 13)`.



Path Compression: Another Clever Idea

Draw the tree after we call `isConnected(14, 13)`.



By compressing the tree with each union and isConnected call, we keep the tree nice and short.

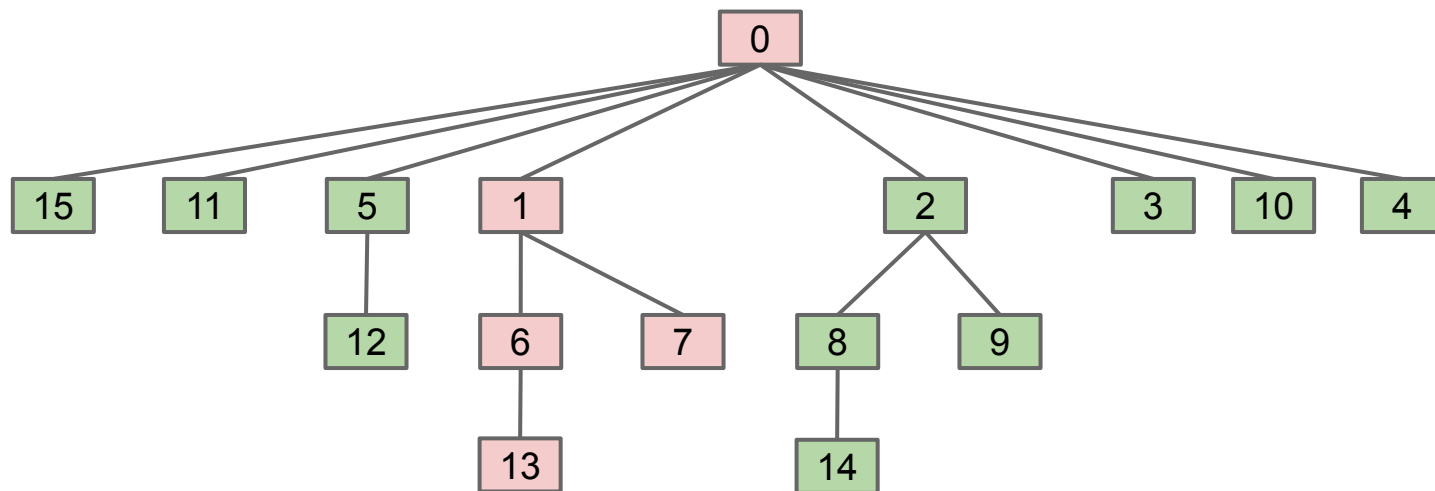
- As number of nodes N grows, our tree tends to get taller.
- As number of operations M grows, our tree tends to get shorter.
 - For enough operations tree height will shrink to 1.

Intuition

By compressing the tree with each union and isConnected call, we keep the tree nice and short.

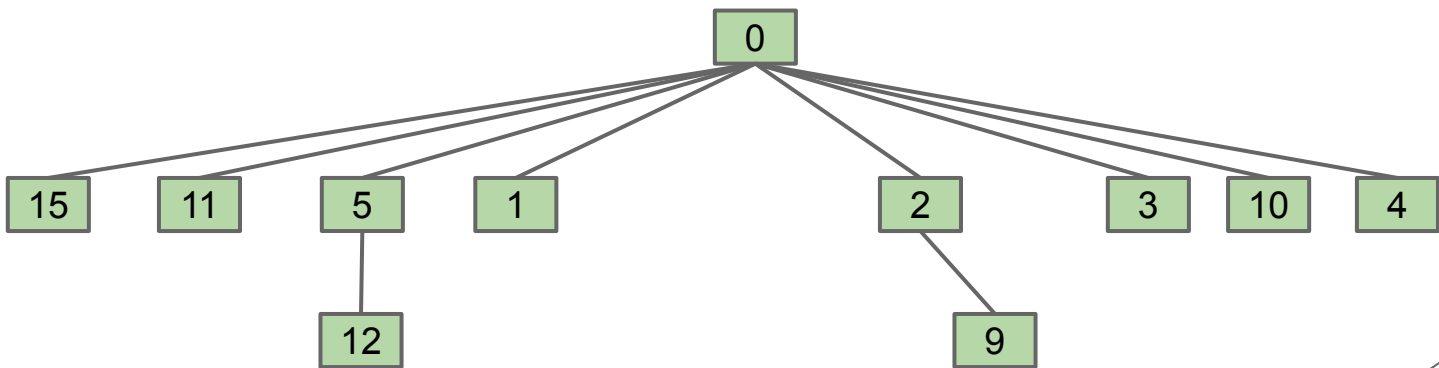
Note: The tree we started with in the exercise you completed is impossible to generate if we're using path compression!

- The structure in red is impossible (try to convince yourself if you'd like).



In CS170, you'll show that each `isConnected` or `connect` operation takes on average $\lg^* N$ time because the tree is kept so compressed.

- $\lg^*$: How many times you need to press the $\log_2$ button on a calculator before you get to a number that is 1 or less. Example:
<http://joshhug/logstar/demo.html>
- M operations on N nodes takes $O(M \lg^* N)$ time for large M .
- $\lg^*$ is less than or equal to 5 for any realistic input.



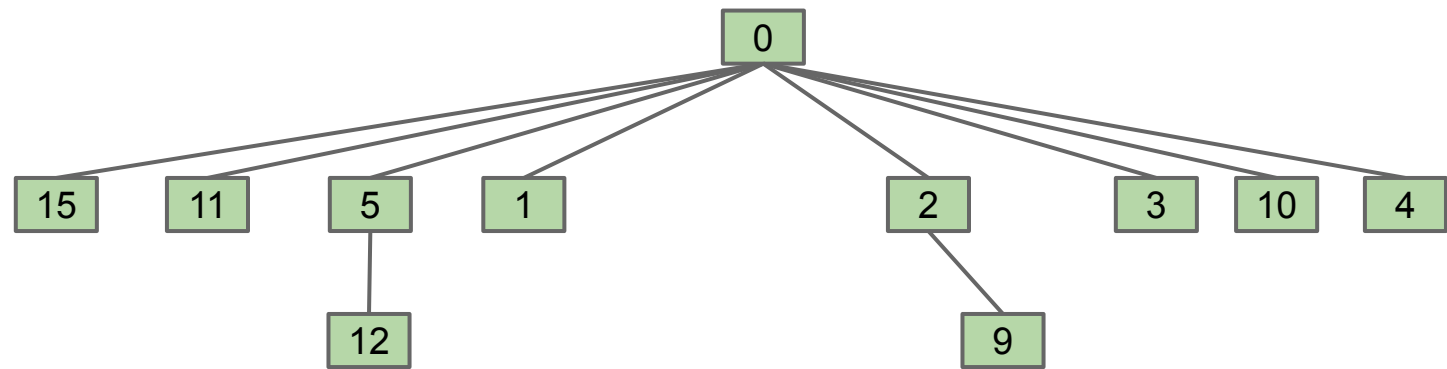
N	$\lg^* N$
1	0
2	1
4	2
16	3
65536	4
2^{65536}	5

2^{16}

Even More Careful Analysis

You can provide an even tighter bound, showing that each operation takes on average $\alpha(N)$ time.

- α is the inverse Ackermann function.
- See “Efficiency of a Good But Not Linear Set Union Algorithm.”
 - Written by Bob Tarjan while at UC Berkeley in 1975.



N	$\alpha(N)$
1	0
...	1
...	2
...	3
...	4
	5

$2^{2^{2^{\dots^2}}}$
65536

And we're done! The end result of our iterative design process is the standard way disjoint sets are implemented today - quick union and path compression.

The ideas that made our implementation efficient:

- Represent sets as connected components (don't track individual connections).
 - **ListOfSetsDS**: Store connected components as a List of Sets (slow, complicated).
 - **QuickFindDS**: Store connected components as set ids.
 - **QuickUnionDS**: Store connected components as parent ids.
 - **WeightedQuickUnionDS**: Also track the size of each set, and use size to decide on new tree root.
 - **WeightedQuickUnionWithPathCompressionDS**: On calls to connect and isConnected, set parent id to the root for all items seen.

Performance Summary

Implementation	Runtime
ListOfSetsDS	$O(NM)$
QuickFindDS	$\Theta(NM)$
QuickUnionDS	$O(NM)$
WeightedQuickUnionDS	$O(M \log N)$
WeightedQuickUnionDSWithPathCompression	$O(M \alpha(N))$

Runtime all based on time to climb tree, which we improved

Runtimes are given assuming:

- We have a DisjointSets object of size N .
- We perform M operations, where an operation is defined as either a call to `connected` or `isConnected`.